

Proyecto_Space_Invaders_v0

July 4, 2026

1 Actividad - Proyecto práctico

La actividad se desarrollará en grupos pre-definidos de 4 alumnos. Se debe indicar los nombres en orden alfabético (de apellidos). Recordad que esta actividad se corresponde con un 30% de la nota final de la asignatura. Se debe entregar el trabajo en la presente notebook.

- Alumno 1: Daniel Alfonso Correa Angarita
- Alumno 2: Cesar Daniel Meneses Guevara
- Alumno 3: America Garrido Sánchez

Colab: [Proyecto Space Invaders v0](#)

Git Hub: [Link del Repositorio](#)

Evidencias de entrenamiento: En esta [carpeta compartida de google drive](#) se guardan los pesos de cada modelo, videos, logs, y el ipynb de este proyecto.

Videos Agente Jugando:

- [Video Modelo DQN Clasico](#)
- [Video Modelo DQN Spatial](#)
- [Video Modelo DQN Dueling](#)

Nota. Se hicieron varias sesiones de trabajo (Peer Programming) actualizando el colab, puede verificar el historial de revisiones en colab para lo mencionado.

1.1 PARTE 1 - Instalación y requisitos previos

Las prácticas han sido preparadas para poder realizarse en el entorno de trabajo de Google Colab. Sin embargo, esta plataforma presenta ciertas incompatibilidades a la hora de visualizar la renderización en gym. Por ello, para obtener estas visualizaciones, se deberá trasladar el entorno de trabajo a local. Por ello, el presente dossier presenta instrucciones para poder trabajar en ambos entornos. Siga los siguientes pasos para un correcto funcionamiento: 1. **LOCAL:** Preparar el enviroment, siguiendo las intrucciones detalladas en la sección *1.1.Preparar enviroment*. 2. **AMBOS:** Modificar las variables “mount” y “drive_mount” a la carpeta de trabajo en drive en el caso de estar en Colab, y ejecturar la celda *1.2.Localizar entorno de trabajo*. 3. **COLAB:** se deberá ejecutar las celdas correspondientes al montaje de la carpeta de trabajo en Drive. Esta

corresponde a la sección 1.3. *Montar carpeta de datos local*. 4. **AMBOS:** Instalar las librerías necesarias, siguiendo la sección 1.4. *Instalar librerías necesarias*.

1.1.1 1.1. Preparar enviroment (solo local)

Para preparar el entorno de trabajo en local, se han seguido los siguientes pasos: 1. En Windows, puede ser necesario instalar las C++ Build Tools. Para ello, siga los siguientes pasos. Alternativamente puedes utilizar WSL2: <https://towardsdatascience.com/how-to-install-openai-gym-in-a-windows-environment-338969e24d30>. 2. Instalar Anaconda 3. Siguiendo el código que se presenta comentado en la próxima celda: Crear un enviroment, cambiar la ruta de trabajo, e instalar librerías básicas.

```
conda create --name miar_rl python=3.8
conda activate miar_rl
cd "PATH_TO_FOLDER"
conda install git
pip install jupyter
```

4. Abrir la notebook con *jupyter-notebook*.

```
jupyter-notebook
```

1.1.2 1.2. Localizar entorno de trabajo: Google colab o local

```
[ ]: # ATENCIÓN!! Modificar ruta relativa a la práctica si es distinta (drive_root)
mount='/content/gdrive'
drive_root = mount + "/My Drive/08_MIAR/actividades/proyecto practico"

try:
    from google.colab import drive
    IN_COLAB=True
except:
    IN_COLAB=False
```

1.1.3 1.3. Montar carpeta de datos local (solo Colab)

```
[ ]: import os
if IN_COLAB:
    print("We're running Colab")

if IN_COLAB:
    print("Colab: mounting Google drive on ", mount)

drive.mount(mount)
```

```

create_drive_root = True
if create_drive_root:
    print("\nColab: making sure ", drive_root, " exists.")
    os.makedirs(drive_root, exist_ok=True)

    print("\nColab: Changing directory to ", drive_root)
    %cd $drive_root
%pwd
print("Archivos en el directorio: ")
print(os.listdir())

```

We're running Colab

Colab: mounting Google drive on /content/gdrive

Mounted at /content/gdrive

Colab: making sure /content/gdrive/My Drive/08_MIAR/actividades/proyecto practico exists.

Colab: Changing directory to /content/gdrive/My

Drive/08_MIAR/actividades/proyecto practico

/content/gdrive/My Drive/08_MIAR/actividades/proyecto practico

Archivos en el directorio:

['viejo entrenamiento', 'dqn clasico', 'dqn spartial', 'Copia entrenamiento', 'dqn dueling', 'Proyecto_práctico.ipynb']

1.1.4 1.4. Instalar librerías necesarias

```

[ ]: if IN_COLAB:
    !pip uninstall jax jaxlib ml-dtypes -y
    %pip install --upgrade ml_dtypes==0.5.0

    %pip install tensorflow==2.18.0
    %pip install tf-keras==2.18.0
    %pip install gym==0.17.3
    %pip install git+https://github.com/Kojoley/atari-py.git
    %pip install keras-rl2==1.0.5

    print(f"Ecosistema clásico cargado en la nube de Google.")

else:
    """
    %pip install gym==0.17.3
    %pip install git+https://github.com/Kojoley/atari-py.git
    %pip install pygame==1.5.0
    %pip install h5py==3.1.0
    """

```

```
%pip install Pillow==9.5.0
%pip install keras-rl2==1.0.5
%pip install Keras==2.2.4
%pip install tensorflow==2.5.3
%pip install torch==2.0.1
%pip install agents==1.4.0
"""
%pip install numpy pandas matplotlib seaborn tensorflow==2.18.0
```

1.2 PARTE 2. Enunciado

Consideraciones a tener en cuenta:

- El entorno sobre el que trabajaremos será el indicado en el listado correspondien de cada grupo y el algoritmo que usaremos será *DQN*.
- Para nuestro ejercicio, el requisito mínimo será alcanzado cuando el agente consiga una **media de recompensa por encima de los puntos indicados en el listado por grupos en modo test**. Por ello, esta media de la recompensa se calculará a partir del código de test en la última celda del notebook.

Este proyecto práctico consta de tres partes:

1. Implementar la red neuronal que se usará en la solución
2. Implementar las distintas piezas de la solución DQN y probar al menos 3 propuestas diferentes de mejora.
3. Justificar la respuesta en relación a los resultados obtenidos e incluir al menos 3 gráficas relevantes comparando las 3 propuestas.

Rúbrica: Se valorará la originalidad en la solución aportada, así como la capacidad de discutir los resultados de forma detallada. El requisito mínimo servirá para aprobar la actividad, bajo premisa de que la discusión del resultado sera apropiada.

IMPORTANTE:

- Si no se consigue una puntuación óptima, responder sobre la mejor puntuación obtenida.
- Para entrenamientos largos, recordad que podéis usar checkpoints de vuestros modelos para retomar los entrenamientos. En este caso, recordad cambiar los parámetros adecuadamente (sobre todo los relacionados con el proceso de exploración).
- Se deberá entregar unicamente el notebook y los pesos del mejor modelo en un fichero .zip, de forma organizada.
- Cada alumno deberá de subir la solución de forma individual.

1.2.1 SpaceInvaders-v0

En nuestro caso es necesario realizar el entrenamiento por refuerzo en el ambiente en el entorno de **SpaceInvaders-v0** la documentación se encuentra en https://ale.farama.org/environments/space_invaders/.

Le objetivo en nuestro caso es: - Alcanzar más de 20 puntos con reward clipping durante más de 100 episodios consecutivos

1.3 PARTE 3. Desarrollo y preguntas

Importar librerías

```
[ ]: import os
os.environ['TF_USE_LEGACY_KERAS']="1"

import tensorflow.keras as tf
import tensorflow as tf1

from keras import __version__
tf.__version__ = __version__

print(tf1.__version__)
print(tf1.config.list_physical_devices('GPU'))
print(tf1.config.list_logical_devices('GPU'))
```

2.18.0

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

[LogicalDevice(name='/device:GPU:0', device_type='GPU')]

```
[ ]: from __future__ import division

# -----

import sys
import os

# PARCHE DE COMPATIBILIDAD K2 PARA TENSORFLOW 2.18 + KERAS-RL2
import tensorflow.keras.models as keras_models
import tf_keras
from tf_keras.models import model_from_config
keras_models.model_from_config = model_from_config
sys.modules['tensorflow.keras.models'].model_from_config = model_from_config

if 'keras.models' in sys.modules:
    sys.modules['keras.models'].model_from_config = model_from_config

print("Parche 'model_from_config' inyectado con éxito usando tf-keras.")

# -----

from PIL import Image
import numpy as np
```

```

import gym
import csv
import os

from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense, Activation, Flatten, Conv2D,
↳Permute, SpatialDropout2D, Input, Lambda

from tensorflow.keras.optimizers.legacy import Adam
import tensorflow.keras.backend as K

from rl.agents.dqn import DQNAgent
from rl.policy import LinearAnnealedPolicy, BoltzmannQPolicy, EpsGreedyQPolicy
from rl.memory import SequentialMemory
from rl.core import Processor
from rl.callbacks import FileLogger, ModelIntervalCheckpoint, Callback

print("Todos los módulos y agentes de Keras-RL2 cargados correctamente")

```

Parche 'model_from_config' inyectado con éxito usando tf-keras.
 Todos los módulos y agentes de Keras-RL2 cargados correctamente

```

[ ]: import os
import json
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

def graph_training(logs_path, model_name, variables=['mean_q']):
    """
    Lee los logs en JSON y genera una cuadrícula 1x3 con las variables
    ↳solicitadas.
    """
    with open(logs_path, 'r') as f:
        data = json.load(f)

    # Crear una figura con 1 fila y 3 columnas, ajustando el tamaño
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    fig.suptitle(f"Rendimiento de Entrenamiento: {model_name}", fontsize=16,
    ↳y=1.05)

    for i, var in enumerate(variables):
        if var in data:
            # Extraemos los datos para la variable actual
            df_var = pd.DataFrame(data[var]).dropna()

            # Graficamos en el sub-gráfico correspondiente (axes[i])

```

```

df_var.plot(ax=axes[i], legend=False, color='#1f77b4') # Azul
↳ clásico

# Configuramos títulos y etiquetas
axes[i].set_title(f"Evolución de {var}")
axes[i].set_xlabel('Steps')
axes[i].set_ylabel(var)
axes[i].grid(True, linestyle='--', alpha=0.7) # Cuadrícula para
↳ mejor lectura
else:
    print(f"Advertencia: La variable '{var}' no se encontró en
↳ {logs_path}")

# Ajusta el espaciado para que no se superpongan los textos
plt.tight_layout()

# Manejo de rutas y guardado
# os.path.dirname(logs_path) obtiene la carpeta donde está el JSON
path_imgs = os.path.join(os.path.dirname(logs_path), "imgs")
os.makedirs(path_imgs, exist_ok=True)

save_path = os.path.join(path_imgs, f"{model_name}_metricas_cuadrícula.png")
plt.savefig(save_path, bbox_inches='tight')
plt.show() # Muestra la gráfica en el notebook antes de continuar

def graph_test_results(test_metrics, model_name):
    """
    Recibe el diccionario de resultados del test y genera una cuadrícula 1x3.
    """
    rewards = test_metrics['episode_reward']
    steps = test_metrics['nb_steps']
    episodios = range(1, len(rewards) + 1)

    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    fig.suptitle(f"Resultados de Evaluación (Test): {model_name}", fontsize=16,
↳ y=1.05)

    # 1. Gráfica de Recompensas a lo largo de los episodios
    axes[0].plot(episodios, rewards, marker='o', linestyle='-', color='#2ca02c')
    axes[0].set_title("Rewards por Episodio")
    axes[0].set_xlabel("Número de Episodio")
    axes[0].set_ylabel("Reward")
    axes[0].grid(True, linestyle='--', alpha=0.7)

    # 2. Gráfica de Supervivencia (Pasos por episodio)
    axes[1].plot(episodios, steps, marker='s', linestyle='-', color='#ff7f0e')

```

```

axes[1].set_title("Supervivencia (Pasos por Episodio)")
axes[1].set_xlabel("Número de Episodio")
axes[1].set_ylabel("Cantidad de Steps")
axes[1].grid(True, linestyle='--', alpha=0.7)

# 3. Diagrama de Caja (Boxplot) para ver la estabilidad
caja = axes[2].boxplot(rewards, vert=True, patch_artist=True)
caja['boxes'][0].set_facecolor('#1f77b4') # Color azul de relleno
axes[2].set_title("Distribución de Rewards")
axes[2].set_ylabel("Reward")
axes[2].set_xticks([1])
axes[2].set_xticklabels([f'{len(rewards)} Episodios'])
axes[2].grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()

```

Configuración base

```

[ ]: BASE_CONFIG = {
    # Entorno y preprocesamiento
    'ENV_NAME': 'SpaceInvaders-v0',
    'WINDOW_LENGTH': 4,
    'INPUT_SHAPE': (84, 84),
    'SEED': 123,

    # Hiperparámetros del agente
    'learning_rate': 0.00025,
    'discount_factor': 0.99,
    'batch_size': 32,
    'target_update_freq': 5000,
    'train_frequency': 4,
    'learning_starts': 10000,

    # Exploración
    'epsilon_start': 1.0,
    'epsilon_end': 0.1,
    'epsilon_decay_steps': 750000,
    'epsilon_test': 0.05,

    # Memoria
    'memory_size': 800000,

    # Entrenamiento
    'max_steps': 1000000,
    'optimizer': 'adam',
    'gradient_clip': 1.0,

```

```
}
```

```
[ ]: from gym import Wrapper
import numpy as np

class FixedFrameSkip(Wrapper):
    def __init__(self, env, skip=BASE_CONFIG['WINDOW_LENGTH']):
        super(FixedFrameSkip, self).__init__(env)
        self._skip = skip
        if hasattr(self.env.unwrapped, 'ale'):
            self.env.unwrapped.frameskip = 1

    def step(self, action):
        total_reward = 0.0
        done = False
        info = {}

        for i in range(self._skip):
            obs, reward, done, info = self.env.step(action)
            total_reward += reward
            if done:
                break

        return obs, total_reward, done, info
```

```
[ ]: env_base = gym.make(BASE_CONFIG['ENV_NAME'])
env = FixedFrameSkip(env_base, skip=4)

np.random.seed(BASE_CONFIG["SEED"])
env.seed(BASE_CONFIG["SEED"])
nb_actions = env.action_space.n
print(f"Acciones disponibles: {str(nb_actions)}" )
print(f"Formato de las observaciones: {env.observation_space}")
```

Acciones disponibles: 6

Formato de las observaciones: Box(0, 255, (210, 160, 3), uint8)

```
[ ]: class AtariProcessor(Processor):
    def process_observation(self, observation):
        assert observation.ndim == 3
        img = Image.fromarray(observation)
        img = img.resize(BASE_CONFIG["INPUT_SHAPE"], Image.LANCZOS).convert('L')
        processed_observation = np.array(img)
        assert processed_observation.shape == BASE_CONFIG["INPUT_SHAPE"]
        return processed_observation.astype('uint8')

    def process_state_batch(self, batch):
```

```

        processed_batch = batch.astype('float32') / 255.
        return processed_batch

    def process_reward(self, reward):
        return np.clip(reward, -1., 1.)

```

```

[ ]: def create_memory():
    return SequentialMemory(
        limit=BASE_CONFIG['memory_size'],
        window_length=BASE_CONFIG['WINDOW_LENGTH']
    )

```

```

[ ]: def create_policy():
    return LinearAnnealedPolicy(
        EpsGreedyQPolicy(),
        attr='eps',
        value_max=BASE_CONFIG['epsilon_start'],
        value_min=BASE_CONFIG['epsilon_end'],
        value_test=BASE_CONFIG['epsilon_test'],
        nb_steps=BASE_CONFIG['epsilon_decay_steps']
    )

```

```

[ ]: def create_agent(model, nb_actions, enable_double=False):
    return DQNAgent(
        model=model,
        nb_actions=nb_actions,
        memory=create_memory(),
        nb_steps_warmup=BASE_CONFIG['learning_starts'],
        target_model_update=BASE_CONFIG['target_update_freq'],
        policy=create_policy(),
        processor=AtariProcessor(),
        batch_size=BASE_CONFIG['batch_size'],
        gamma=BASE_CONFIG['discount_factor'],
        train_interval=BASE_CONFIG['train_frequency'],
        enable_double_dqn=enable_double,
    )

```

```

[ ]: def compile_agent(agent):
    agent.compile(
        optimizer=Adam(
            learning_rate=BASE_CONFIG['learning_rate'],
            clipnorm=BASE_CONFIG['gradient_clip']
        ),
        metrics=['mae']
    )
    return agent

```

```
[ ]: import csv
from rl.callbacks import Callback
import os

class MetricLogger(Callback):
    def __init__(self, version, base_path):
        self.version = version
        self.folder = os.path.join(base_path, f"version_{version}")
        os.makedirs(self.folder, exist_ok=True)
        self.file_path = os.path.join(self.folder, "metrics.csv")
        self.fields = ['episode', 'steps', 'reward']

        archivo_existe = os.path.exists(self.file_path) and os.path.
        ↪getsize(self.file_path) > 0
        modo = 'a' if archivo_existe else 'w'

        with open(self.file_path, modo, newline='') as f:
            if not archivo_existe:
                writer = csv.DictWriter(f, fieldnames=self.fields)
                writer.writeheader()

    def on_episode_end(self, episode, logs):
        row = {
            'episode': episode,
            'steps': logs.get('nb_steps'),
            'reward': logs.get('episode_reward')
        }

        with open(self.file_path, 'a', newline='') as f:
            writer = csv.DictWriter(f, fieldnames=self.fields)
            writer.writerow(row)
```

```
[ ]: class SaveWeightsInterval(Callback):
    def __init__(self, agent, version, interval, base_path="."):
        self.agent = agent
        self.version = version
        self.interval = interval
        self.folder = os.path.join(base_path, f"version_{version}", "pesos")
        os.makedirs(self.folder, exist_ok=True)

    def on_episode_end(self, episode, logs={}):
        if (episode % self.interval == 0):
            filename = os.path.join(self.folder, f"dqn_v_{self.
            ↪version}_weights_{episode}_episodes.h5f")
            self.agent.save_weights(filename, overwrite=True)
            print(f"\n[Callback] Pesos guardados en: {filename}")
```

```
[ ]: from rl.callbacks import Callback
import os

class SaveBestWeights(Callback):
    def __init__(self, agent, version, base_path):
        self.agent = agent
        self.version = version
        self.best_reward = -float('inf')
        self.folder = os.path.join(base_path, f"version_{version}",
        ↪ "mejor_modelo")
        os.makedirs(self.folder, exist_ok=True)

    def on_episode_end(self, episode, logs={}):
        current_reward = logs.get('episode_reward')

        if current_reward is not None and current_reward > self.best_reward:
            self.best_reward = current_reward
            filename = os.path.join(self.folder, "best_weights.h5f")
            self.agent.save_weights(filename, overwrite=True)
            print(f"\nNuevo récord: {current_reward}. Pesos guardados en
            ↪ {filename}")

[ ]: class TotalStepStopper(Callback):
    def __init__(self, max_steps):
        super(TotalStepStopper, self).__init__()
        self.max_steps = max_steps

    def on_step_end(self, step, logs={}):
        if self.model.step >= self.max_steps:
            print(f"\n[Stopper] ¡Meta global alcanzada ({self.model.step} >=
            ↪ {self.max_steps})! Deteniendo...")
            self.model.stop_training = True

[ ]: import os
import glob
import re
import pandas as pd

def entrenando_modelo_con_recuperacion(agent, env, version, max_steps,
    ↪ carpeta_guardado, callbacks_lista, ruta_csv):
    """
    Busca los pesos más recientes, lee el CSV para saber el progreso exacto y
    ↪ continúa el entrenamiento.
    """
    os.makedirs(carpeta_guardado, exist_ok=True)

    # 1. Buscar los últimos pesos
```

```

patron_busqueda = os.path.join(carpeta_guardado, f"version_{version}",
↪ "pesos", f"dqn_v_{version}_weights*_episodes.h5f.index")
archivos = glob.glob(patron_busqueda)

archivo_reciente_base = None
max_episodio = -1

for archivo in archivos:

    match = re.search(r'weights_(\d+)_episodes', archivo)

    if match:
        episodio = int(match.group(1))
        if episodio > max_episodio:
            max_episodio = episodio
            archivo_reciente_base = archivo[:-6]

steps_reales_completados = 0

if archivo_reciente_base:
    print(f";Guardado encontrado! Restaurando pesos desde el episodio_
↪ {max_episodio}: {archivo_reciente_base}")
    agent.load_weights(archivo_reciente_base)

    try:
        if os.path.exists(ruta_csv):
            df = pd.read_csv(ruta_csv)
            steps_reales_completados = int(df['steps'].max())
            print(f"CSV leído correctamente. El modelo ya ha entrenado_
↪ {steps_reales_completados} steps.")
        else:
            print("No se encontró el CSV. Asumiendo 0 steps (Peligro de_
↪ sobreentrenamiento).")
        except Exception as e:
            print(f"Error al leer el CSV: {e}. Asumiendo 0 steps.")

    else:
        print("No se encontraron pesos previos. Iniciando desde cero.")

steps_restantes = max_steps - steps_reales_completados

if steps_restantes <= 0:
    print(f"El agente ya cumplió o superó la meta de los {max_steps} steps.
↪ ")
    return None

```

```

    print(f"Retomando entrenamiento. Faltan exactamente: {steps_restantes}␣
↪steps.")

    historial = agent.fit(
        env,
        nb_steps=steps_restantes,
        visualize=False,
        verbose=2,
        callbacks=callbacks_lista
    )

    return historial

```

```

[ ]: import os
import shutil
import glob
import numpy as np
from gym import wrappers

def evaluar_agente(agent, env, folder_path, video_dir=None, nb_episodes=5):
    weights_filename = os.path.join(folder_path, 'best_weights.h5f')
    agent.load_weights(weights_filename)
    agent.test_policy.eps = 0.0

    grabar_video = video_dir is not None
    local_video_folder = "/content/video_eval_temporal" if grabar_video else␣
↪None

    if grabar_video:
        env_ejecucion = wrappers.Monitor(
            env,
            local_video_folder,
            force=True,
            video_callable=lambda episode_id: True
        )
    else:
        env_ejecucion = env

    print(f"-> Iniciando evaluación ({'Modo Video Activo' if grabar_video else␣
↪'Modo Métricas Puro'})...")
    results = agent.test(env_ejecucion, nb_episodes=nb_episodes,␣
↪visualize=False, verbose=0)

    if grabar_video:
        env_ejecucion.close()
        os.makedirs(video_dir, exist_ok=True)
        for archivo in glob.glob(os.path.join(local_video_folder, "*")):

```

```

        shutil.copy(archivo, video_dir)
        print(f"Video exportado exitosamente a: {video_dir}")

    return results

def print_results(agent_name, results):
    rewards = results.history['episode_reward']

    print(f"Resultados del Agente: {agent_name}")
    print(f"Rendimiento promedio: {np.mean(rewards)}")
    print(f"Rendimiento mínimo: {np.min(rewards)}")
    print(f"Rendimiento máximo: {np.max(rewards)}")
    print(f"Desviación estándar: {np.std(rewards)}")

```

1. Implementación de la red neuronal

```

[ ]: def create_dqn_model(input_shape, window_length, nb_actions,
    ↪with_spatial_dropout = False):
    input_layer = Input(shape=(window_length,) + input_shape)

    print(K.image_data_format())
    if K.image_data_format() == 'channels_last':
        x = Permute((2, 3, 1))(input_layer)
    elif K.image_data_format() == 'channels_first':
        x = Permute((1, 2, 3))(input_layer)
    else:
        raise RuntimeError('Unknown image_dim_ordering.')

    x = Conv2D(32, (8, 8), strides=(4, 4), padding='valid')(x)
    x = Activation('relu')(x)
    if with_spatial_dropout:
        x = SpatialDropout2D(0.1)(x)

    x = Conv2D(64, (4, 4), strides=(2, 2), padding='valid')(x)
    x = Activation('relu')(x)
    if with_spatial_dropout:
        x = SpatialDropout2D(0.1)(x)

    x = Conv2D(64, (3, 3), strides=(1, 1), padding='valid')(x)
    x = Activation('relu')(x)
    if with_spatial_dropout:
        x = SpatialDropout2D(0.1)(x)

    x = Flatten()(x)
    x = Dense(512)(x)
    x = Activation('relu')(x)

```

```

output_layer = Dense(nb_actions)(x)
output_layer = Activation('linear')(output_layer)

model = Model(inputs=input_layer, outputs=output_layer)
return model

```

```

[ ]: import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Conv2D, Activation,
    ↳ SpatialDropout2D, Layer
import tensorflow.keras.backend as K

# Definimos la operación matemática de Dueling como una Capa segura
@tf.keras.utils.register_keras_serializable()
class DuelingOutputLayer(Layer):
    def __init__(self, **kwargs):
        super(DuelingOutputLayer, self).__init__(**kwargs)

    def call(self, inputs):
        value_stream, advantage_stream = inputs
        # Aplicamos la fórmula:  $Q(s,a) = V(s) + (A(s,a) - \text{mean}(A(s,a)))$ 
        mean_advantage = tf.reduce_mean(advantage_stream, axis=1, keepdims=True)
        return value_stream + (advantage_stream - mean_advantage)

def create_dueling_dqn_model(input_shape, window_length, nb_actions,
    ↳ with_spatial_dropout=False):
    input_layer = Input(shape=(window_length,) + input_shape)

    if K.image_data_format() == 'channels_last':
        x = tf.keras.layers.Permute((2, 3, 1))(input_layer)
    else:
        x = tf.keras.layers.Permute((1, 2, 3))(input_layer)

    x = Conv2D(32, (8, 8), strides=(4, 4), padding='valid',
    ↳ activation='relu')(x)
    if with_spatial_dropout:
        x = SpatialDropout2D(0.1)(x)

    x = Conv2D(64, (4, 4), strides=(2, 2), padding='valid',
    ↳ activation='relu')(x)
    if with_spatial_dropout:
        x = SpatialDropout2D(0.1)(x)

    x = Conv2D(64, (3, 3), strides=(1, 1), padding='valid',
    ↳ activation='relu')(x)
    if with_spatial_dropout:
        x = SpatialDropout2D(0.1)(x)

```

```

x = Flatten()(x)

value_stream = Dense(512, activation='relu')(x)
value_stream = Dense(1, activation='linear')(value_stream)

advantage_stream = Dense(512, activation='relu')(x)
advantage_stream = Dense(nb_actions, activation='linear')(advantage_stream)

outputs = DuelingOutputLayer(name='dueling_output')([value_stream,
↳advantage_stream])

model = Model(inputs=input_layer, outputs=outputs,
↳name='Dueling_DQN_SpaceInvaders')

return model

```

2. Implementación de la solución DQN A continuación se realizan las 3 implementaciones solicitadas: - Implementación 1: **DQN Clásico (Baseline)** - Implementación 2: **DQN con SpatialDropout** - Implementación 3: **Dueling DQN**

Implementación 1: DQN Clásico (Baseline)

```

[ ]: dqn_model_without_spatial_dropout =
↳create_dqn_model(input_shape=BASE_CONFIG["INPUT_SHAPE"],
↳window_length=BASE_CONFIG["WINDOW_LENGTH"],
nb_actions=nb_actions)

print(dqn_model_without_spatial_dropout.summary())

baseline_agent = create_agent(model=dqn_model_without_spatial_dropout,
↳nb_actions=nb_actions)
baseline_agent = compile_agent(baseline_agent)

DQN_CLASICO_VERSION_ACTUAL = "1.0"
DQN_CLASICO_CARPETA = '/content/gdrive/MyDrive/08_MIAR/actividades/proyecto_
↳practico/dqn clasico/'
DQN_CLASICO_METRICS =
↳f'{DQN_CLASICO_CARPETA}version_{DQN_CLASICO_VERSION_ACTUAL}/metricas.csv'
DQN_CLASICO_LOGS = f'/content/gdrive/MyDrive/08_MIAR/actividades/proyecto_
↳practico/dqn clasico/version_{DQN_CLASICO_VERSION_ACTUAL}/logs.json'

```

```

channels_last
Model: "model_5"

```

Layer (type)	Output Shape	Param #
input_4 (InputLayer)	[(None, 4, 84, 84)]	0
permute_3 (Permute)	(None, 84, 84, 4)	0
conv2d_9 (Conv2D)	(None, 20, 20, 32)	8224
activation_10 (Activation)	(None, 20, 20, 32)	0
conv2d_10 (Conv2D)	(None, 9, 9, 64)	32832
activation_11 (Activation)	(None, 9, 9, 64)	0
conv2d_11 (Conv2D)	(None, 7, 7, 64)	36928
activation_12 (Activation)	(None, 7, 7, 64)	0
flatten_3 (Flatten)	(None, 3136)	0
dense_8 (Dense)	(None, 512)	1606144
activation_13 (Activation)	(None, 512)	0
dense_9 (Dense)	(None, 6)	3078
activation_14 (Activation)	(None, 6)	0
Total params: 1687206 (6.44 MB)		
Trainable params: 1687206 (6.44 MB)		
Non-trainable params: 0 (0.00 Byte)		
None		

```
[ ]: dqn_clasico_guardar_pesos = SaveWeightsInterval(
    agent=baseline_agent,
    version=DQN_CLASICO_VERSION_ACTUAL,
    interval=160,
    base_path=DQN_CLASICO_CARPETA
)
dqn_clasico_guardar_mejor_modelo = SaveBestWeights(
    agent=baseline_agent,
    version=DQN_CLASICO_VERSION_ACTUAL,
    base_path=DQN_CLASICO_CARPETA
)
dqn_clasico_metricas = MetricLogger(
```

```

        version=DQN_CLASICO_VERSION_ACTUAL,
        base_path=DQN_CLASICO_CARPETA)

dqn_clasico_lista_callbacks = [dqn_clasico_guardar_pesos,
                                dqn_clasico_guardar_mejor_modelo,
                                dqn_clasico_metricas,
                                FileLogger(DQN_CLASICO_LOGS, interval=160),
                                ↵
                                ↵TotalStepStopper(max_steps=BASE_CONFIG['max_steps'])]

history_1 = entrenando_modelo_con_recuperacion(
    agent=baseline_agent,
    env=env,
    version=DQN_CLASICO_VERSION_ACTUAL,
    max_steps=BASE_CONFIG['max_steps'],
    carpeta_guardado=DQN_CLASICO_CARPETA,
    callbacks_lista=dqn_clasico_lista_callbacks,
    ruta_csv=DQN_CLASICO_METRICS
)

```

¡Guardado encontrado! Restaurando pesos desde el episodio 3200:
/content/gdrive/MyDrive/08_MIAR/actividades/proyecto practico/dqn
clasico/version_1.0/pesos/dqn_v_1.0_weights_3200_episodes.h5f
CSV leído correctamente. El modelo ya ha entrenado 32718 steps.
Retomando entrenamiento. Faltan exactamente: 967282 steps.
Training for 967282 steps ...
done, took 0.549 seconds

```

[ ]: # Pasamos la carpeta raíz donde están guardados los archivos de la versión ↵
      ↵clásica
DQN_CLASICO_VERSION_ACTUAL = "1.0"
CARPETA_CHECKPOINT_CLASICO = f'/content/gdrive/MyDrive/08_MIAR/actividades/
      ↵proyecto practico/dqn clasico/version_{DQN_CLASICO_VERSION_ACTUAL}/
      ↵mejor_modelo'
CARPETA_VIDEO_CLASICO = f'/content/gdrive/MyDrive/08_MIAR/actividades/proyecto ↵
      ↵practico/dqn clasico/version_{DQN_CLASICO_VERSION_ACTUAL}/video'

# Ejecucion del test con 100 episodios para metricas.
clasico_results = evaluar_agente(
    agent=baseline_agent,
    env=env,
    folder_path=CARPETA_CHECKPOINT_CLASICO,
    nb_episodes=100
)

# Generación de video.
clasico_video = evaluar_agente(

```


channels_last
Model: "model_2"

Layer (type)	Output Shape	Param #
input_2 (InputLayer)	[(None, 4, 84, 84)]	0
permute_1 (Permute)	(None, 84, 84, 4)	0
conv2d_3 (Conv2D)	(None, 20, 20, 32)	8224
activation_5 (Activation)	(None, 20, 20, 32)	0
spatial_dropout2d_3 (SpatialDropout2D)	(None, 20, 20, 32)	0
conv2d_4 (Conv2D)	(None, 9, 9, 64)	32832
activation_6 (Activation)	(None, 9, 9, 64)	0
spatial_dropout2d_4 (SpatialDropout2D)	(None, 9, 9, 64)	0
conv2d_5 (Conv2D)	(None, 7, 7, 64)	36928
activation_7 (Activation)	(None, 7, 7, 64)	0
spatial_dropout2d_5 (SpatialDropout2D)	(None, 7, 7, 64)	0
flatten_1 (Flatten)	(None, 3136)	0
dense_2 (Dense)	(None, 512)	1606144
activation_8 (Activation)	(None, 512)	0
dense_3 (Dense)	(None, 6)	3078
activation_9 (Activation)	(None, 6)	0

Total params: 1687206 (6.44 MB)
Trainable params: 1687206 (6.44 MB)
Non-trainable params: 0 (0.00 Byte)

None

```
[ ]: # Entrenamiento
# Prepararamos los callbacks contruidos
dqn_spatial_guardar_pesos = SaveWeightsInterval(
    agent=spatial_dropout_agent,
    version=DQN_SPATIAL_VERSION_ACTUAL,
    interval=160,
    base_path = DQN_SPATIAL_CARPETA)

dqn_spatial_guardar_mejor_modelo = SaveBestWeights(
    agent=spatial_dropout_agent,
    version=DQN_SPATIAL_VERSION_ACTUAL,
    base_path=DQN_SPATIAL_CARPETA)

dqn_spatial_metricas = MetricLogger(
    version=DQN_SPATIAL_VERSION_ACTUAL,
    base_path=DQN_SPATIAL_CARPETA)

dqn_spatial_lista_callbacks = [dqn_spatial_guardar_pesos,
                                dqn_spatial_guardar_mejor_modelo,
                                dqn_spatial_metricas,
                                FileLogger(DQN_SPATIAL_LOGS, interval=160),
                                ↵
                                ↪TotalStepStopper(max_steps=BASE_CONFIG['max_steps'])]

history_2 = entrenando_modelo_con_recuperacion(
    agent=spatial_dropout_agent,
    env=env,
    version=DQN_SPATIAL_VERSION_ACTUAL,
    max_steps=BASE_CONFIG['max_steps'],
    carpeta_guardado=DQN_SPATIAL_CARPETA,
    callbacks_lista=dqn_spatial_lista_callbacks,
    ruta_csv=DQN_SPATIAL_METRICS
)
```

```
[ ]: # Evaluacion - spatial_dropout_agent
CARPETA_CHECKPOINT_SPATIAL = '/content/gdrive/MyDrive/08_MIAR/actividades/
    ↪proyecto practico/dqn spatial/version_1.0/mejor_modelo'
CARPETA_VIDEO_SPATIAL = '/content/gdrive/MyDrive/08_MIAR/actividades/proyecto_
    ↪practico/dqn spatial/version_1.0/video'

# Ejecucion del test con 100 episodios para metricas.
spatial_results = evaluar_agente(
    agent=spatial_dropout_agent,
    env=env,
    folder_path=CARPETA_CHECKPOINT_SPATIAL,
    nb_episodes=100
)
```

```
# Generación de video.
spatial_video = evaluar_agente(
    agent=spatial_dropout_agent,
    env=env,
    folder_path=CARPETA_CHECKPOINT_SPATIAL,
    video_dir=CARPETA_VIDEO_SPATIAL,
    nb_episodes=2
)

print_results("DQN SPATIAL", spatial_results)
```

-> Iniciando evaluación (Modo Métricas Puro)...

```
/usr/local/lib/python3.12/dist-packages/tf_keras/src/engine/training_v1.py:2354:
UserWarning: `Model.state_updates` will be removed in a future version. This
property should not be used in TensorFlow 2.0, as `updates` are applied
automatically.
```

```
updates=self.state_updates,
```

-> Iniciando evaluación (Modo Video Activo)...

Video exportado exitosamente a:

```
/content/gdrive/MyDrive/08_MIAR/actividades/proyecto practico/dqn
spartial/version_1.0/video
```

Resultados del Agente: DQN SPATIAL

Rendimiento promedio: 9.3

Rendimiento mínimo: 4.0

Rendimiento máximo: 28.0

Desviación estándar: 4.468780594300866

Implementación 3: Dueling DQN

```
[ ]: dueling_dqn_model = create_dueling_dqn_model(input_shape=BASE_CONFIG["INPUT_SHAPE"],
    window_length=BASE_CONFIG["WINDOW_LENGTH"],
    nb_actions = nb_actions)
print(dueling_dqn_model.summary())

dueling_agent = create_agent(model=dueling_dqn_model, nb_actions=nb_actions,
    enable_double=True)
dueling_agent = compile_agent(dueling_agent)

DQN_DUELING_VERSION_ACTUAL = "1.0"
DQN_DUELING_CARPETA = '/content/gdrive/MyDrive/08_MIAR/actividades/proyecto
    practico/dqn dueling/'
DQN_DUELING_METRICS = f'/content/gdrive/MyDrive/08_MIAR/actividades/proyecto
    practico/dqn dueling/version_{DQN_DUELING_VERSION_ACTUAL}/metricas.csv'
```

```
DQN_DUELING_LOGS = f'/content/gdrive/MyDrive/08_MIAR/actividades/proyecto_
↳ practico/dqn dueling/version_{DQN_DUELING_VERSION_ACTUAL}/logs.json'
```

Model: "Dueling_DQN_SpaceInvaders"

```
-----
Layer (type)                Output Shape                Param #   Connected to
=====
input_3 (InputLayer)        [(None, 4, 84, 84)]        0         []

permute_2 (Permute)         (None, 84, 84, 4)         0         ['input_3[0][0]']

conv2d_6 (Conv2D)           (None, 20, 20, 32)         8224      ['permute_2[0][0]']

conv2d_7 (Conv2D)           (None, 9, 9, 64)          32832     ['conv2d_6[0][0]']

conv2d_8 (Conv2D)           (None, 7, 7, 64)          36928     ['conv2d_7[0][0]']

flatten_2 (Flatten)         (None, 3136)               0         ['conv2d_8[0][0]']

dense_4 (Dense)              (None, 512)                1606144   ['flatten_2[0][0]']

dense_6 (Dense)              (None, 512)                1606144   ['flatten_2[0][0]']

dense_5 (Dense)              (None, 1)                  513       ['dense_4[0][0]']

dense_7 (Dense)              (None, 6)                  3078      ['dense_6[0][0]']

dueling_output (DuelingOut  (None, 6)                  0         ['dense_5[0][0]',
putLayer)
'dense_7[0][0]']

=====
Total params: 3293863 (12.57 MB)
Trainable params: 3293863 (12.57 MB)
```

Non-trainable params: 0 (0.00 Byte)

None

```
[ ]: # Entrenamiento
# Prepararemos los callbacks contruidos
dqn_dueling_guardar_pesos = SaveWeightsInterval(
    agent=dueling_agent,
    version=DQN_DUELING_VERSION_ACTUAL,
    interval=160,
    base_path=DQN_DUELING_CARPETA)

dqn_dueling_guardar_mejor_modelo = SaveBestWeights(
    agent=dueling_agent,
    version=DQN_DUELING_VERSION_ACTUAL,
    base_path=DQN_DUELING_CARPETA)

dqn_dueling_metricas = MetricLogger(
    version=DQN_DUELING_VERSION_ACTUAL,
    base_path=DQN_DUELING_CARPETA)

dqn_dueling_lista_callbacks = [dqn_dueling_guardar_pesos,
                                dqn_dueling_guardar_mejor_modelo,
                                dqn_dueling_metricas,
                                FileLogger(DQN_DUELING_LOGS, interval=160),
                                ↵
                                ↪TotalStepStopper(max_steps=BASE_CONFIG['max_steps'])]

history_3 = entrenando_modelo_con_recuperacion(
    agent=dueling_agent,
    env=env,
    version=DQN_DUELING_VERSION_ACTUAL,
    max_steps=BASE_CONFIG['max_steps'],
    carpeta_guardado=DQN_DUELING_CARPETA,
    callbacks_lista=dqn_dueling_lista_callbacks,
    ruta_csv=DQN_DUELING_METRICS
)

[ ]: CARPETA_CHECKPOINT_DUELING = '/content/gdrive/MyDrive/08_MIAR/actividades/
    ↪proyecto practico/dqn dueling/version_1.0/mejor_modelo'
CARPETA_VIDEO_DUELING = '/content/gdrive/MyDrive/08_MIAR/actividades/proyecto_
    ↪practico/dqn dueling/version_1.0/video'

# Ejecucion del test con 100 episodios para metricas.
dueling_results = evaluar_agente(
    agent=dueling_agent,
```

```

    env=env,
    folder_path=CARPETA_CHECKPOINT_DUELING,
    nb_episodes=100
)

# Generación de video.
dueling_video = evaluar_agente(
    agent=dueling_agent,
    env=env,
    folder_path=CARPETA_CHECKPOINT_DUELING,
    video_dir=CARPETA_VIDEO_DUELING,
    nb_episodes=2
)

print_results("DQN DUELING", dueling_results)

```

-> Iniciando evaluación (Modo Métricas Puro)...

```

/usr/local/lib/python3.12/dist-packages/tf_keras/src/engine/training_v1.py:2354:
UserWarning: `Model.state_updates` will be removed in a future version. This
property should not be used in TensorFlow 2.0, as `updates` are applied
automatically.

```

```

    updates=self.state_updates,

```

-> Iniciando evaluación (Modo Video Activo)...

Video exportado exitosamente a:

```

/content/gdrive/MyDrive/08_MIAR/actividades/proyecto practico/dqn
dueling/version_1.0/video

```

Resultados del Agente: DQN DUELING

Rendimiento promedio: 20.4

Rendimiento mínimo: 9.0

Rendimiento máximo: 36.0

Desviación estándar: 4.8207883172775805

3. Justificación de los parámetros seleccionados y de los resultados obtenidos

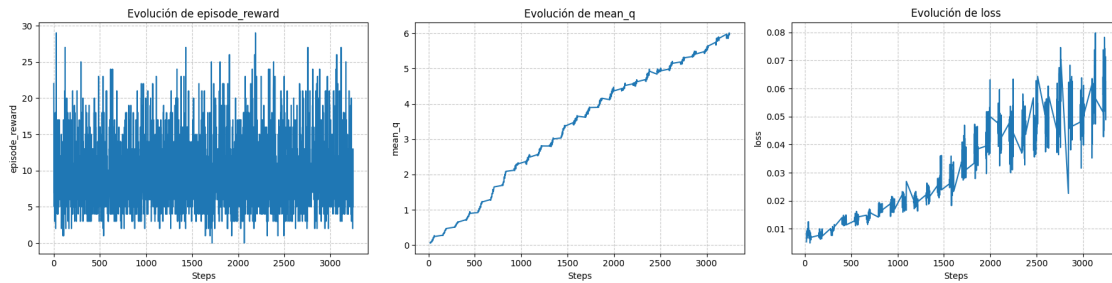
DQN Clasica

```

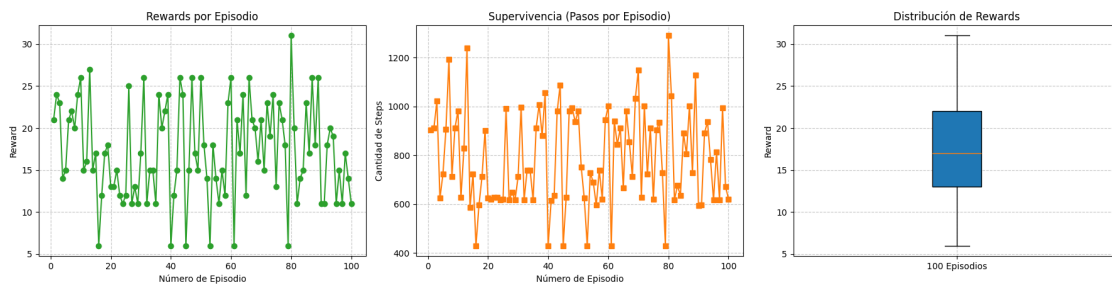
[ ]: graph_training(DQN_CLASICO_LOGS,model_name= "DQN_Clasico",
    ↪variables=["episode_reward", "mean_q", "loss"])
graph_test_results(clasico_results.history, model_name= "DQN_Clasico")
print("")
print_results("DQN Clásico", clasico_results)

```

Rendimiento de Entrenamiento: DQN_Clasico



Resultados de Evaluación (Test): DQN_Clasico



Resultados del Agente: DQN Clásico

Rendimiento promedio: 17.24

Rendimiento mínimo: 6.0

Rendimiento máximo: 31.0

Desviación estándar: 5.729083696368906

Conclusiones

Luego de trabajar con el primer modelo (DQN Clásico) en el entorno SpaceInvaders-v0 podemos decir que la arquitectura de tres capas convolucionales, logro establecer un punto referencia en torno a la puntuacion, a pesar de que el agente no alcanzo a llegar al reto propuesto de recompensa media de 20 puntos en 100 episodios, quedo cerca con 17.24 puntos y una desviacion estandar de 5.72, nuestro agente logro picos de puntajes de 31.0 puntos en la fase de pruebas.

Pudimos notar, debido a la desviacion estandar, que nuestro agente presenta un poco de inestabilidad por lo que depende mucho de las configuraciones iniciales que utilizamos.

Estuvimos investigando un poco y vimos que mantener la red neuronal con el parámetro **with_spatial_dropout = False** resultó ser un factor clave (Ya lo veremos en el modelo 2). En los entornos visuales de Atari, donde entidades críticas (como los misiles enemigos) ocupan apenas unos pocos píxeles en la pantalla, evitar el “apagado” aleatorio de los mapas de características convolucionales hizo que el agente mantuviera una visión completa del entorno, permitiéndole reaccionar a tiempo y asegurar el rendimiento observado.

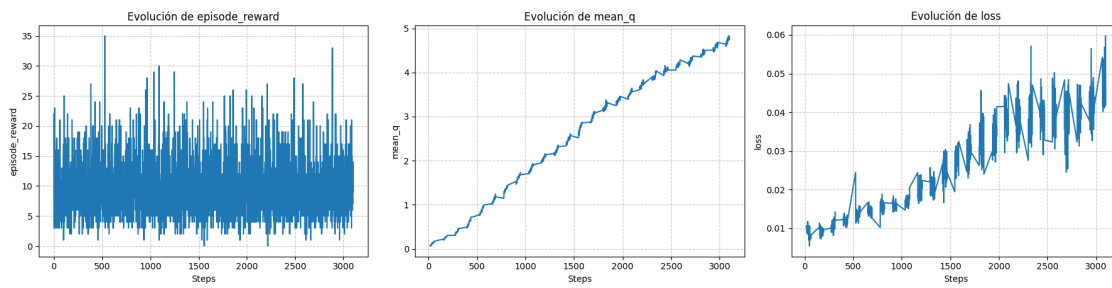
Si observamos las graficas de entrenamiento, podremos ver la inestabilidad del agente en la puntuacion, asi mismo, en la funcion de perdidas a medida que iban aumentando los steps, la perdida subia, lo que nos hace concluir dicha inestabilidad, tal vez por falta de entrenamiento.

En las graficas de test, podemos ver el diagrama de bigotes, donde si ejecutaramos y ejecutaramos muchas veces el agente con los 100 episodios probablemente en alguna ocasion obtendriamos una media de mas de 20 puntos.

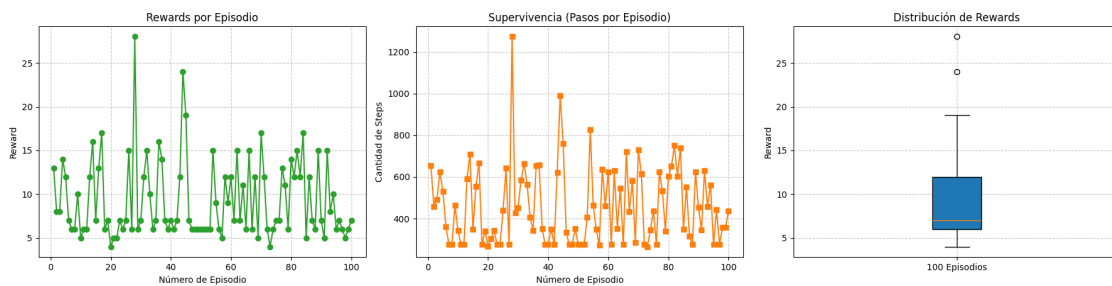
DQN Spatial

```
[ ]: graph_training(DQN_SPATIAL_LOGS,model_name= "DQN_Spatial",
    ↪variables=["episode_reward", "mean_q", "loss"])
graph_test_results(spatial_results.history, model_name= "DQN_Spatial")
print("")
print_results("DQN Spatial", spatial_results)
```

Rendimiento de Entrenamiento: DQN_Spatial



Resultados de Evaluación (Test): DQN_Spatial



Resultados del Agente: DQN Spatial

Rendimiento promedio: 9.3

Rendimiento mínimo: 4.0

Rendimiento máximo: 28.0

Desviación estándar: 4.468780594300866

Conclusiones

Luego de realizar la ejecución del segundo modelo implementado con **with_spatial_dropout = True**, podemos observar una degradación en el rendimiento, obteniendo una recompensa media de 9.3 puntos (Por debajo del objetivo de 20) con una desviación estándar (std_reward=4.46) con un máximo puntaje 28.0, esto nos confirma lo que mencionamos en el primer modelo, al tener en True el parámetro de dropout, la inestabilidad aumenta, si hemos entrenado el modelo con pocos steps.

Luego de realizar investigación y viendo el entorno de Space Invaders, esta caída se justifica teóricamente por la incompatibilidad de esta técnica (el dropout = True) de regularización con entornos espacialmente sensibles:

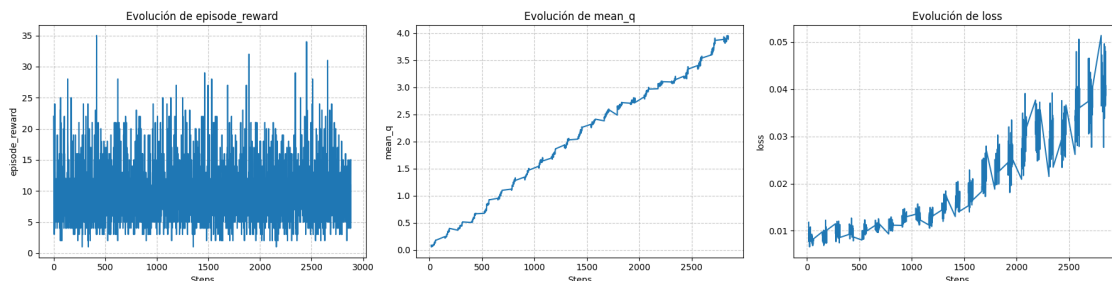
1. **Ceguera inducida:** El Spatial Dropout desactiva mapas de características completos. En nuestro entorno, las amenazas críticas como los misiles ocupan muy pocos píxeles por lo que apagar mapas enteros durante el entrenamiento deja a la red neuronal literalmente “ciega” ante estos peligros.
2. **Corrupción del valor Q:** Al no poder percibir las amenazas de forma constante, el agente sufre muertes que considera impredecibles. Esto corrompe la estimación matemática de la función de valor (Q) y genera una política errática, impidiendo que el algoritmo converja hacia una estrategia de evasión estable.

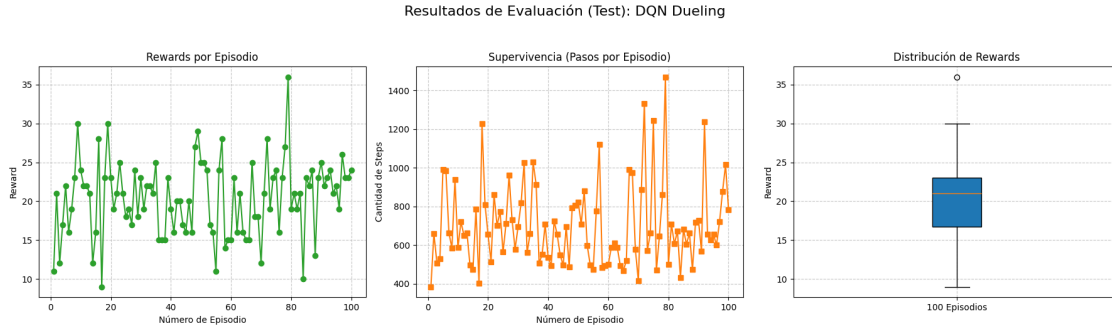
Si analizamos las graficas de entrenamiento vemos la alta variabilidad de recompensas, al tener el dropout en true, hace que nuestro agente no pueda obtener toda la informacion del entorno en la cantidad de steps de entrenamiento seleccionada, lo que afecta de gran forma la evaluacion con 100 episodios en etapa de evaluacion, en el diagrama de bigotes podemos ver que la recompensa varia por debajo de 20 puntos, que fue el reto propuesto. Para mejorar esto luego de una investigacion realizada, es necesario entrenar por muchos mas steps el modelo para lograr una recompensa media decente.

DQN Dueling

```
[ ]: graph_training(DQN_DUELING_LOGS, model_name= "DQN Dueling",  
    ↪variables=["episode_reward", "mean_q", "loss"])  
graph_test_results(dueling_results.history, model_name= "DQN Dueling")  
print("")  
print_results("DQN Dueling", dueling_results)
```

Rendimiento de Entrenamiento: DQN Dueling





Resultados del Agente: DQN Dueling

Rendimiento promedio: 20.4

Rendimiento mínimo: 9.0

Rendimiento máximo: 36.0

Desviación estándar: 4.8207883172775805

Conclusiones

En la evaluación del tercer modelo (Dueling DQN) superó el umbral exigido obteniendo una recompensa media de (20.4 puntos). Su característica más notable es la alta varianza: presenta una desviación estándar elevada (4.82) y logró un pico máximo de 36.0 puntos, superando las rachas del modelo clásico.

Este comportamiento bimodal se explica por la separación interna de la red en dos flujos $V(s)$ y $A(s,a)$:

Picos de rendimiento máximo: Al separar el cálculo de “qué tan bueno es el estado” de “qué tan buena es la acción”, la red puede identificar de forma general los momentos de peligro sin tener que aprender el valor de cada movimiento individualmente. Esto le permitió alcanzar episodios con un rendimiento táctico (36 puntos).

Alta dispersión (Inestabilidad): En Space Invaders, casi todos los estados requieren una acción muy específica e inmediata para sobrevivir (esquivar un misil). La separación de las capas V y A introduce una mayor complejidad de optimización matemática que, con 1.6 millones de steps, no logró estabilizarse al 100%. Esto provocó que el agente alternara entre partidas magistrales y episodios con errores tempranos.

Justificación & Conclusiones Generales A continuación se detalla cada apartado.

Justificación:

Cabe mencionar que el entrenamiento de los tres modelos se realizó con aproximadamente 1.600.000 steps, cada uno de ellos demora aproximadamente 7 horas entrenando, queríamos realizar dicho entrenamiento igual para todos los modelos con el fin de realizar una comparación.

Nos encontramos con dificultades a la hora de la configuración del ambiente por las incompatibilidades de versiones que se habían mencionado en clase, sin embargo, realizando investigación

podimos sortear dichos inconvenientes, además, tuvimos algunos retos en la elección de los hiperparametros porque al principio nuestros modelos no estaban entrenando correctamente, nos dimos cuenta de esto porque las métricas de perdida (loss) y media de Q (mean_q) no se estaban actualizando luego de muchos steps (perdimos dias de entrenamiento, debido a este problema), cuando aprendimos de estas experiencias tomamos la decisión de entrenar con 1.600.000 a pesar de que en algunos artículos que vimos la recomendación era intentar hacerlo con más steps, a pesar de esto, fue gratificante ver que nuestros modelos pudieron cumplir con nuestras expectativas.

La configuracion base para los tres modelos es la siguiente:

```
BASE_CONFIG = {
    # Entorno y preprocesamiento
    'ENV_NAME': 'SpaceInvaders-v0',
    'WINDOW_LENGTH': 4,
    'INPUT_SHAPE': (84, 84),
    'SEED': 123,

    # Hiperparámetros del agente
    'learning_rate': 0.00025,
    'discount_factor': 0.99,
    'batch_size': 32,
    'target_update_freq': 5000,
    'train_frequency': 4,
    'learning_starts': 10000,

    # Exploración
    'epsilon_start': 1.0,
    'epsilon_end': 0.1,
    'epsilon_decay_steps': 750000,
    'epsilon_test': 0.05,

    # Memoria
    'memory_size': 800000,

    # Entrenamiento
    'max_steps': 1000000,
    'optimizer': 'adam',
    'gradient_clip': 1.0,
}
```

A pesar de que la visualizacion del entorno de atari es en si un rectangulo, realizamos una investigacion para ajustar la dimension de las imagenes que tendria el modelo como entrada, a principio habiamos decidido mantener una configuracion rectangular, sin embargo, realizando investigacion decidimos optar por un cuadrado que era una configura mas estandar del entorno (INPUT_SHAPE: (84, 84) y WINDOW_LENGTH: 4). Asi mismo, tuvimos en cuenta que al juntar cuatro frames consecutivos en escala de grises, ibamos a garantizar que el agente no solo perciba posiciones estáticas, sino que pueda inferir la velocidad y la trayectoria de los elementos clave (como el movimiento de los alienígenas y los proyectiles).

Para garantizar que el modelo no converja prematuramente hacia policies suboptimas locales se

decidio que el parametro `epsilon_decay_steps` decaiga al 75% del entrenamiento total, ademas, el agente mantuvo un 10% por exploracion en la fase final asegurando que siga descubriendo estados importantes en fases avanzadas del juego.

Intentamos que la combinación de la frecuencia de actualizacion de la red objetivo (`target_update_freq: 5000`) y el recorte de gradientes (`gradient_clip: 1.0`) constituyera un mecanismo de defensa contra la inestabilidad al Q-Learning profundo, a pesar de esto, vimos que aun asi tuvimos gran inestabilidad en nuestros modelos.

En esto punto, tuvimos un gran problema con el tema de memoria, en la primera tanda de ejecucion de entrenamientos, la memoria estaba alrededor de 200.000, haciendo que nuestros modelos no entrenaran, por lo que tomamos la decision de aumentarla a 800.000 transiciones (`memory_size`) que nos proporciono un balance entre la diversidad de datos y consumo de memoria RAM. En el periodo de calentamiento de 10.000 pasos (`learning_starts`), aseguramos que el optimizador Adam (`learning_rate: 0.00025`) comience a trabajar sobre lotes (`batch_size: 32`) de datos completamente descorrelacionados, rompiendo la dependencia temporal de las secuencias de juego y estabilizando la función de perdida.

En la parte de entrenamiento decidimos establecer el límite en 1.000.000 de pasos (`max_steps`), sin embargo, nuestros modelos entrenaron aproximadamente 1.600.000 steps lo cual permitio un marco de ejecución predecible que evito ciclos de entrenamiento infinitos o desgastes innecesarios, permitiendo una comparación justa y equitativa en el mismo marco de tiempo computacional para todas los modelos evaluadas.

Conclusiones Generales:

Luego de ahondar en todo el trabajo realizado podemos decir que cualquier elecccion errada en los hiperparametros puede ocasionar perdida de tiempo y de recursos computacionales (como nos paso en esta ocasion). Aprendimos con este taller que el entrenamiento de los modelos requieren gran precision, para que los modelos puedan converger a optimos globales, la conclusion que sacamos de nuestro trabajo es que el entrenamiento con 1.600.000 steps se hizo insuficiente para lograr estabilizar por completo a los tres modelos, el que mas se acerco a esto fue el modelo de Dueling, demostrando su versatilidad y adaptacion con pocos steps.

Mientras que el DQN Clasico tendio a sobreajustarse a trayectorias de exito especificas, la regularización introducida por el Spatial Dropout exige un mayor esfuerzo de entrenamiento inicial(necesitaba mas steps para estabilizarse) pero recompensa con policies mas resistentes a las variaciones espaciales del entorno. Esto no quiere decir que sea peor que el DQN Clasico, solo que necesita mas tiempo de entrenamiento para estabilizar las recompensas.

A diferencia del DQN estándar o con dropout, el enfoque Dueling descompone el problema. Esta bifurcacion del flujo de datos en la red neuronal justifica su uso al estabilizar las estimaciones de Q, mitigando la sobreestimacion que a menudo limita el rendimiento a largo plazo en los agentes DQN clasicos.

Para alcanzar y mantener las metricas por encima de los 20 puntos de forma sostenida durante 100 episodios, es necesario intentar entrenar los modelos con mas steps. Los picos que observamos en las versiones mejoradas (Spatial y Dueling) nos confirman que modificar la arquitectura de la red neuronal impacta directamente en la capacidad del agente para superar la barrera de rendimiento del modelo clásico.

Por ultimo, podemos decir que el objetivo principal (reto) del proyecto fue cumplido con el modelo

de Dueling debido a que mantuvo una media sobre 20 puntos.
